FECAP - FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação
6º Semestre - Turma 6º CCOMP

SMASH

RELATÓRIO DE STATUS E MITIGAÇÃO DE RISCOS

Entrega 2 - Redes de Computadores e Cibersegurança

Projeto 7 - Grupo Smash

Felipe Vallim Soares - 24026060
Guilhermy Mariano Lisboa Garcia - 23025371
Gustavo Oliveira Demetrio - 24026213
Saulo Pereira de Jesus - 24026095

Prof. Victor Bruno Alexander Rosetti

São Paulo
2026

IDENTIFICAÇÃO DO DOCUMENTO

Quadro 1 - Identificação do relatório.

Campo	Informação
Projeto	Smash - parede instrumentada de treino de tênis (Projeto 7)
Disciplina	Redes de Computadores e Cibersegurança - Entrega 2
Data	06/11/2026
Versão do documento	2.1 (a versão 1.0 é o Plano da Entrega 1; a 2.1 acrescenta a análise do Aikido)
Documento anterior	Plano de Gerenciamento de Riscos, versão 1.0 (Entrega 1, 25/09/2026)
Repositório	*https://github.com/2026-2-NCC6/Projeto7*
Código de referência	Tag *entrega2-ciberseguranca-aikido*, pasta *src/Entrega1*. A Entrega 2 continuou o código da Entrega 1, na mesma pasta; a tag marca o estado entregue agora, e os caminhos citados são relativos a essa pasta
Código-fonte	Acompanha este relatório. Também pode ser baixado em *https://github.com/2026-2-NCC6/Projeto7/archive/refs/tags/entrega2-ciberseguranca-aikido.zip*

SUMÁRIO

1 INTRODUÇÃO....4
2 METODOLOGIA DA VERIFICAÇÃO....6
3 STATUS E RESPOSTAS AOS RISCOS....8
4 ESTRATÉGIAS DE RESPOSTA....11
5 MATRIZES DE PROBABILIDADE E IMPACTO....12
6 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO....14
7 ANÁLISE DO REPOSITÓRIO COM O AIKIDO....21
8 EVIDÊNCIAS DE TESTES DE SEGURANÇA....24
9 RISCOS MATERIALIZADOS E NOVOS RISCOS....25
10 COBERTURA DOS REQUISITOS DE SEGURANÇA....27
11 LIMITAÇÕES DA VERIFICAÇÃO....29
12 RISCO RESIDUAL E PRÓXIMOS PASSOS....30
13 MONITORAMENTO E REVISÃO....30
APROVAÇÃO....31
ANEXO A - REGISTRO DE RISCOS ATUALIZADO....32
ANEXO B - SAÍDAS DOS TESTES....35
ANEXO C - CAPTURAS DO AIKIDO....38

1 INTRODUÇÃO

Este relatório complementa o Plano de Gerenciamento de Riscos entregue na Entrega 1. Naquele documento, a equipe levantou 19 riscos no código e no projeto do Smash e definiu, para cada um, uma resposta e um prazo. A maior parte tinha prazo na Entrega 2 e estava com status "Não iniciado". Este relatório mostra o que foi corrigido no código, onde está cada correção, como ela foi testada e o que ainda falta.

A Entrega 2 da disciplina pede a entrega complementar dos relatórios e as mitigações implementadas no código do sistema. Por isso, este relatório usa a mesma numeração da Entrega 1 (R01 a R19): cada risco aparece com o que mudou desde então, e os riscos novos começam em R20.

O Plano da Entrega 1 assumiu três compromissos para esta revisão: implementar e verificar no código as respostas aos riscos Extremos (R01, R02 e R03), associar cada resposta concluída ao commit que a implementa e repetir a verificação de dependências com npm audit. As respostas aos três riscos Extremos estão implementadas e testadas no código, mas ainda dependem de um último passo fora dele: publicar a API (R01) e gravar o firmware novo na placa (R02 e R03). Os outros dois compromissos foram cumpridos e estão nas seções 6 e 8 e nos Anexos A e B.

Depois da primeira versão deste relatório, o professor orientou analisar o repositório em uma plataforma on-line. A equipe usou o Aikido Security e conferiu no código cada um dos 8 achados (seção 7). Dois viraram riscos novos (R29 e R30), três alertas de dependências entraram no R11 e três correções pequenas foram aplicadas.

1.1 O que mudou no projeto desde a Entrega 1

O hardware mudou durante o desenvolvimento, e algumas descrições do Plano da Entrega 1 ficaram desatualizadas. O Quadro 2 registra as mudanças que afetam a leitura dos riscos.

Quadro 2 - Mudanças no projeto entre as entregas.

Aspecto	Entrega 1	Entrega 2
Parede	Nove alvos em matriz 3 × 3	Seis alvos em grade 3 × 2
Leitura dos piezos	Multiplexador de canais	Cada piezo no seu próprio pino ADC1 do ESP32

Aspecto	Entrega 1	Entrega 2
Integração MQTT	Prevista (R13)	Fora do escopo. O enunciado cita MQTT só como uma das opções de comunicação; o Smash usa WebSocket entre parede e aplicativo e HTTP entre aplicativo e API
Atuador	Não havia	LED da placa (GPIO 2), acionado pela parede e pelo aplicativo
Registro da sessão	O aplicativo enviava o resultado pronto	O servidor abre a sessão antes do jogo e aceita um único resultado por sessão (R04)

1.2 Resumo executivo

Dos 30 riscos registrados (19 da Entrega 1 e 11 novos), 13 estão concluídos, 5 têm a correção pronta no código e só dependem de publicar a API ou gravar a placa, 2 foram encerrados, 5 seguem em andamento, 4 foram aceitos com justificativa e 1 ainda não começou. Na Entrega 1 havia 3 riscos Extremos e 10 Altos; depois dos passos que faltam, nenhum fica Extremo e só o R14, que é físico, continua Alto.

2 METODOLOGIA DA VERIFICAÇÃO

Na Entrega 1, os riscos foram levantados por revisão estática do código. Nesta entrega, cada correção foi verificada por uma evidência que pode ser repetida, como o próprio Plano exigia: a eficácia de uma resposta é medida por commit, configuração ou teste, e não pela intenção declarada. Também foi feita uma nova rodada de busca de vulnerabilidades. Foram usadas sete formas de verificação:

- **Testes automatizados:** cada correção veio com testes de unidade. O backend tem 86 testes e o aplicativo tem 117, e todos passam no commit de referência.
- **Script de testes de segurança:** *npm run security:test* faz contra a API e a parede os ataques de um cliente modificado, como senha errada repetida, partida forjada e conexão sem segredo. Foi rodado no emulador da parede e também pode ser apontado para a placa; a saída está no Anexo B.
- **Busca de segredos no histórico:** o gitleaks varreu todos os commits do repositório, e o histórico foi revisado também à procura de senhas escritas no código.
- **Análise estática do código (SAST):** o Semgrep rodou regras de segurança para TypeScript, Node.js, C/C++, JWT e segredos sobre o backend, o aplicativo, o firmware e os scripts.
- **Varredura da API em execução (DAST):** o OWASP ZAP examinou a API rodando localmente, primeiro no modo baseline e depois nas 8 rotas da descrição OpenAPI *scripts/zap-openapi.yaml*, autenticado e com ataques ativos. A API tem 9 rotas; só a saída da conta ficou de fora.
- **Verificação de dependências:** *npm audit --omit=dev* no backend e no aplicativo, antes e depois das correções. O firmware também foi compilado com o arduino-cli para a placa ESP32 Dev Module.
- **Análise do repositório no Aikido:** plataforma on-line que examina o código, as dependências, os segredos e a conta do GitHub. Os 8 achados e a conferência de cada um estão na seção 7.

A severidade residual de cada risco usa as mesmas escalas de probabilidade e impacto e a mesma matriz da Entrega 1 (seções 6 e 7 daquele documento). Cinco riscos estão com a correção concluída no código, mas dependem de um passo fora dele. Para todos eles vale o mesmo critério: a severidade residual indicada é a que fica depois desse passo, e até lá vale a severidade da Entrega 1.

Dois passos ainda não foram feitos: gravar o firmware novo na placa ESP32 e publicar a API com certificado reconhecido (seção 12). Por isso R01, R02, R03, R05 e R07 estão como "Concluído no código", com o passo que falta indicado.

3 STATUS E RESPOSTAS AOS RISCOS

3.1 Resumo do status

O Quadro 3 resume a situação dos 19 riscos da Entrega 1. Na Entrega 1 havia 13 riscos Extremos ou Altos. Deles, 5 estão concluídos (R04, R06, R09, R11 e R17); 5 estão com a correção pronta no código e só falta publicar a API ou gravar a placa (R01, R02, R03, R05 e R07); 1 foi encerrado (R13, porque o MQTT saiu do projeto); e 2 continuam em andamento fora do código (R14, que é físico, e R15, reformulado para o prazo da apresentação).

Quadro 3 - Situação dos riscos da Entrega 1.

Status	Riscos	Total
Concluído	R04, R06, R08, R09, R10, R11, R12 e R17	8
Concluído no código; falta publicar a API ou gravar a placa	R01 (publicar) e R02, R03, R05 e R07 (gravar)	5
Encerrado	R13 (risco eliminado) e R19 (concluído desde a Entrega 1)	2
Em andamento, fora do código	R14, R15 (reformulado), R16 e R18	4

3.2 Tabela de respostas aos riscos

O Quadro 4 reúne a resposta dada a cada risco. "Nível" é a severidade na Entrega 1 ou, nos riscos novos, a classificação do Quadro 11. Os responsáveis são as trilhas do Plano da Entrega 1. Nos prazos, "Apresentação" é a Semana FECAP de Tecnologia, de 17 a 24/11/2026. A severidade residual de cada risco está no Quadro 14.

Quadro 4 - Plano de respostas aos riscos.

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R01 - API em HTTP sem criptografia	Extremo	Mitigar	HTTPS com certificado, HTTP recusado e aplicativo sem http:// fora do desenvolvimento (item 6.1).	Backend e Aplicativo	Apresentação	Concluído no código; falta publicar
R02 - WebSocket aceitava qualquer cliente	Extremo	Mitigar	Segredo de pareamento, comparado sem parar no primeiro caractere diferente, e impactos só para o aplicativo pareado (item 6.2).	Firmware e Aplicativo	Apresentação	Concluído no código; falta gravar a placa
R03 - Senha do Wi-Fi previsível e pública	Extremo	Prevenir	Senha e segredo no *secrets.h*, fora do Git; sem ele o firmware não compila (item 6.3).	Firmware	Apresentação	Concluído no código; falta gravar a placa
R04 - Resultado aceito sem prova	Alto	Mitigar	Sessão aberta pelo servidor, ticket assinado, limites de duração, batidas e pontos e um resultado por ticket (item 6.4).	Backend	Entrega 2	Concluído

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R05 - Qualquer cliente trocava a sessão da parede	Alto	Mitigar	Um aplicativo por vez; só o pareado manda comandos (item 6.5).	Firmware	Apresentação	Concluído no código; falta gravar a placa
R06 - Login e cadastro sem limite de tentativas	Alto	Mitigar	Limite por IP e por conta, com bloqueio e resposta 429 (item 6.6).	Backend	Entrega 2	Concluído
R07 - Parede sem limite de clientes e comandos	Alto	Mitigar	Limites de cliente, de tamanho e de comandos por segundo; ping desconecta quem sumiu (item 6.7).	Firmware	Apresentação	Concluído no código; falta gravar a placa
R08 - CORS aberto e API em todas as interfaces	Médio	Mitigar	CORS desligado por padrão e escuta em 127.0.0.1 (item 6.8).	Backend	Entrega 2	Concluído
R09 - Senha padrão do banco como reserva	Alto	Prevenir	Sem valor padrão: a API não sobe sem usuário e senha (item 6.9).	Backend	Entrega 2	Concluído
R10 - Token de sete dias sem revogação	Médio	Mitigar	Validade de 12 horas, versão de token por usuário e público próprio (item 6.10).	Backend	Entrega 2	Concluído
R11 - Dependências com alertas conhecidos	Alto	Mitigar	NestJS 11, multer fixado, correções compatíveis e alertas não alcançáveis justificados por escrito (itens 6.11 e 7.2).	Backend e Aplicativo	Entrega 2	Concluído
R12 - Pouca trilha de auditoria	Médio	Mitigar	Log em JSON de autenticação e recusas, sem senha nem token (item 6.12).	Backend	Entrega 2	Concluído
R13 - MQTT sem controles definidos	Alto	Prevenir	MQTT retirado do escopo: o risco deixou de existir.	Firmware e Backend	-	Encerrado
R14 - Isolamento mecânico insuficiente	Alto	Mitigar	Validar em etapas (um alvo, dois lado a lado, os seis) e calibrar o limiar de cada alvo.	Firmware	Apresentação	Em andamento
R15 - Tempo para concluir os passos pendentes	Alto	Mitigar	Reformulado: revisão semanal dos passos da seção 12, com os riscos Extremos primeiro.	Gerente de Projeto	Apresentação	Em andamento
R16 - Alteração de critérios ou datas	Médio	Aceitar (ativo)	Relatórios, plano e código no mesmo repositório; datas e formatos confirmados com os professores.	Gerente de Projeto	Contínuo	Em andamento
R17 - Dados pessoais reais em demonstrações	Alto	Mitigar	Comandos para criar e apagar contas fictícias (item 6.13).	Backend e Aplicativo	Entrega 2	Concluído
R18 - Única placa ESP32	Médio	Aceitar (ativo)	Modo simulado do aplicativo e eco Serial como contingência; avaliar uma segunda placa.	Firmware	Contínuo	Em andamento
R19 - Rede do local de demonstração	Médio	Prevenir	A parede cria a própria rede (SoftAP).	Firmware	-	Encerrado

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R20 - Segredo de pareamento no pacote do aplicativo	Baixo	Aceitar (ativo)	Segredo diferente para cada parede, trocado antes de cada apresentação. Ele só serve dentro da rede WPA2 da parede.	Firmware e Aplicativo	Apresentação	Em andamento
R21 - Versões antigas deixam de salvar sessões	Médio	Mitigar	Usar só a versão atual do aplicativo nas demonstrações. O servidor responde com uma mensagem clara: "A sessão precisa ser iniciada antes de ser registrada."	Aplicativo	Entrega 2	Concluído
R22 - Limite de tentativas atrás de proxy	Médio	Prevenir	Variável *TRUST_PROXY=true* no ambiente publicado, para o IP de cada aparelho vir do proxy. Já implementada e documentada no *.env.example*.	Backend	Entrega 2	Concluído
R23 - API sem cabeçalhos de segurança	Muito Baixo	Mitigar	Cabeçalhos de segurança com o helmet e *Cache-Control: no-store* em todas as respostas (item 6.14).	Backend	Entrega 2	Concluído
R24 - API aceitava o segredo JWT de exemplo	Médio	Prevenir	A API não inicia com o segredo de exemplo nem com um segredo curto (item 6.15).	Backend	Entrega 2	Concluído
R25 - Bloqueio do login de outra pessoa	Médio	Aceitar (ativo)	É o custo conhecido de limitar tentativas por conta. O bloqueio é curto (15 minutos) e o limite por endereço impede que alguém bloqueie muitas contas de uma vez.	Backend	-	Aceito
R26 - Segredo de pareamento no endereço da conexão	Baixo	Aceitar (ativo)	A parede não registra o endereço pedido e só é alcançável dentro da própria rede WPA2. Trocar o segredo junto com a senha do Wi-Fi.	Firmware	-	Aceito
R27 - Cadastro revela se o e-mail tem conta	Médio	Aceitar (ativo)	O aviso ajuda quem já tem conta, e o limite de tentativas do R06 impede testar muitos e-mails em pouco tempo. O login não revela se o e-mail existe: a mensagem de erro e o bloqueio são iguais para e-mail com e sem conta (Anexo B.1).	Backend	-	Aceito
R28 - Sem exclusão da conta pelo jogador	Médio	Mitigar	Criar a exclusão da conta pelo próprio jogador. Até lá, a equipe apaga os dados a pedido.	Backend e Aplicativo	Apresentação	Não iniciado
R29 - Comparação do segredo no emulador	Muito Baixo	Mitigar	Comparação com *timingSafeEqual*, como o firmware já fazia (item 6.16).	Firmware	Entrega 2	Concluído
R30 - GitHub sem lista de IPs permitidos	Baixo	Aceitar (ativo)	Recurso exclusivo do GitHub Enterprise Cloud. Usar autenticação em dois fatores nas contas com acesso de escrita (item 7.2).	Gerente de Projeto	Contínuo	Aceito

4 ESTRATÉGIAS DE RESPOSTA

As estratégias são as definidas no Plano da Entrega 1. Esta seção mostra como cada uma foi usada; a resposta de cada risco está no Quadro 4.

4.1 Estratégias para ameaças (riscos negativos)

- **Prevenir:** eliminar a causa do risco. Usada em R03 (senha fora do repositório), R09 (sem senha padrão), R13 (MQTT fora do escopo), R19 (rede própria da parede), R22 e R24 (segredo de exemplo recusado).
- **Mitigar:** reduzir a probabilidade ou o impacto até um nível aceitável. Usada em R01, R02, R04 a R08, R10 a R12, R14, R15, R17, R21, R23, R28 e R29.
- **Transferir:** não foi usada: nenhum risco foi repassado a terceiros. A opção pode aparecer no R01, se a API for publicada em um provedor que cuide do certificado HTTPS.
- **Aceitar (ativo):** manter o risco, com justificativa escrita e contingência. Usada em R16, R18, R20, R25, R26, R27 e R30. Dentro do R11, os alertas não alcançáveis foram aceitos da mesma forma, como o Plano previa.

4.2 Estratégias para oportunidades (riscos positivos)

Nenhuma oportunidade foi registrada neste ciclo. Como ganho da entrega, o script *npm run security:test* passa a repetir os ataques a cada mudança e também poderá ser apontado para a placa real.

5 MATRIZES DE PROBABILIDADE E IMPACTO

As escalas e a matriz são as das seções 6 e 7 do Plano da Entrega 1. O Quadro 5 mostra os riscos antes das respostas, e o Quadro 6 mostra o que sobrou depois delas.

5.1 Antes das respostas

Quadro 5 - Matriz de probabilidade e impacto na Entrega 1.

Probabilidade \ Impacto	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	Médio	Alto	Alto	**Extremo** R02, R03	Extremo
Alta	Baixo	Médio	**Alto** R05, R06	**Alto** R04, R15	**Extremo** R01
Média	Baixo	Médio	**Médio** R08, R10, R12, R16, R19	**Alto** R07, R11, R13, R14, R17	**Alto** R09
Baixa	Muito Baixo	Baixo	Baixo	**Médio** R18	Médio
Muito Baixa	Muito Baixo	Muito Baixo	Baixo	Baixo	Médio

- **Extremo:** 3 riscos (R01, R02 e R03). **Alto:** 10. **Médio:** 6. **Baixo e Muito Baixo:** nenhum.

5.2 Depois das respostas

O Quadro 6 mostra, na mesma matriz, a severidade residual, já com os riscos novos (seção 9).

Quadro 6 - Matriz de probabilidade e impacto com a severidade residual.

Probabilidade \ Impacto	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	Médio	Alto	Alto	Extremo	Extremo
Alta	Baixo	Médio	Alto	Alto	Extremo
Média	Baixo	**Médio** R21, R25, R27	**Médio** R04, R16, R22, R28	**Alto** R14	Alto
Baixa	Muito Baixo	**Baixo** R12	**Baixo** R05, R06, R10, R11, R20, R26, R30	**Médio** R02, R07, R15, R17, R18	**Médio** R01, R09
Muito Baixa	Muito Baixo	**Muito Baixo** R23, R29	**Baixo** R08, R19	**Baixo** R03	**Médio** R24

- **Extremo:** nenhum (eram 3). **Alto:** 1 (eram 10), o R14, que é físico e não se resolve no código.
- **Médio:** 15 (R01, R02, R04, R07, R09, R15, R16, R17, R18, R21, R22, R24, R25, R27 e R28). **Baixo e Muito Baixo:** 13 (R03, R05, R06, R08, R10, R11, R12, R19, R20, R23, R26, R29 e R30).

6 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO

Esta seção descreve cada correção em quatro partes: como era antes, o que mudou, como foi testado e o que ainda falta. O título de cada item mostra a severidade na Entrega 1 e a severidade residual. O commit e os arquivos de cada correção estão no Quadro 15, no Anexo A.

6.1 R01 - A API usava HTTP sem criptografia (Extremo → Médio, falta publicar)

- **Antes:** o aplicativo enviava e-mail, senha e o token de acesso em texto claro (*http://*) para a API, que escutava em todas as interfaces de rede da máquina.
- **O que mudou:** a API passou a servir HTTPS quando recebe um certificado (variáveis *TLS_KEY_PATH* e *TLS_CERT_PATH*). Com certificado, ela não responde mais por HTTP. Fora do modo de desenvolvimento, o aplicativo não inicia se o endereço da API não começar com *https://*.

  Em desenvolvimento, o HTTP continua permitido, para o aplicativo falar com o notebook da equipe na rede de testes.
- **Como testamos:** com um certificado de teste, a API respondeu por HTTPS e recusou a conexão por HTTP na mesma porta (Anexo B). Cinco testes de unidade cobrem a regra do aplicativo.
- **O que falta:** publicar a API em um servidor com certificado reconhecido, por exemplo atrás de um proxy HTTPS. Até a publicação, vale a severidade da Entrega 1.

6.2 R02 - O WebSocket da parede aceitava qualquer cliente (Extremo → Médio, falta gravar)

- **Antes:** qualquer aparelho conectado ao Wi-Fi da parede podia abrir o WebSocket, receber todos os impactos e mandar comandos.
- **O que mudou:** a parede agora exige um segredo de pareamento. O aplicativo conecta em *ws://<parede>:81/?token=<segredo>*; com o segredo errado ou ausente, a parede responde "nao autorizado" e fecha a conexão.

  A comparação do segredo não para no primeiro caractere diferente, para o tempo de resposta não dar pistas sobre o segredo. Impactos e batimentos vão só para o aplicativo pareado, e não mais para todos os conectados.
- **Como testamos:** testes R02 do Anexo B.1, no emulador da parede; o firmware com a mesma regra compila para o ESP32.
- **O que falta:** gravar o firmware na placa e repetir o teste nela. O canal continua sem criptografia (*ws://*), porque o ESP32 não serve *wss://* com folga, e o segredo

vai no próprio endereço da conexão (R26). O que protege o segredo é a rede: a parede cria a própria rede Wi-Fi com WPA2 e senha forte (R03).

6.3 R03 - A senha do Wi-Fi da parede era previsível e pública (Extremo → Baixo, falta gravar)

- **Antes:** a senha da rede da parede ("smash1234") estava escrita em *config.h*, um arquivo versionado em repositório público.
- **O que mudou:** a senha do Wi-Fi e o segredo de pareamento saíram do *config.h* e foram para o *secrets.h*, que está no *.gitignore* e nunca vai para o Git. O repositório só tem o modelo *secrets_example.h*, com valores de exemplo.

  Sem o *secrets.h*, o firmware não compila e mostra como criá-lo. O README do firmware orienta a gerar senha e segredo fortes para cada parede (*openssl rand -base64 18* e *openssl rand -hex 16*) e a trocar os dois antes de cada apresentação.
- **Como testamos:** sem o *secrets.h*, a compilação falha com a mensagem "Falta secrets.h" (Anexo B); com ele, o firmware compila. No código atual, a senha antiga só aparece no aviso do README que proíbe usá-la.
- **O que falta:** a senha antiga continua no histórico do Git, de onde não pode ser apagada, e continua gravada na placa física. Por isso ela é tratada como vazada: a placa precisa ser gravada com o firmware novo e uma senha nova antes da próxima demonstração. Até lá, vale a severidade da Entrega 1.

6.4 R04 - O resultado da partida era aceito sem prova (Alto → Médio)

- **Antes:** o aplicativo enviava o resultado pronto (nível, pontos, acertos e duração), e o backend aceitava qualquer valor dentro de limites largos. Um cliente modificado podia inventar partidas e subir no ranking.
- **O que mudou:** agora o servidor abre a sessão antes do jogo. Ao começar uma partida, o aplicativo chama *POST /me/sessions/start*. O servidor confere se o nível está liberado para o jogador (no máximo um acima do último concluído) e devolve um ticket assinado com o jogador, o modo, o nível e a hora de início.

  O resultado só é aceito com esse ticket. O servidor recusa o resultado quando o ticket é de outro jogador ou foi alterado, quando o modo ou o nível não batem com os do início, ou quando a duração informada é maior que o tempo real desde o início.

  Também recusa quando há mais batidas do que a parede consegue registrar nesse tempo (uma a cada 200 ms, o intervalo mínimo do firmware) e quando a pontuação passa do máximo possível para aquele número de acertos.

Cada ticket vale para um único resultado. Isso é garantido por uma restrição UNIQUE no banco, que funciona mesmo com duas requisições chegando ao mesmo tempo.

- **Como testamos:** testes R04 do Anexo B.1, na API real, mais testes de unidade no backend e no aplicativo.
- **O que falta:** dentro desses limites, um cliente modificado ainda consegue declarar um resultado plausível que não aconteceu. Provar a partida de ponta a ponta exigiria que a própria parede assinasse os impactos, o que fica como melhoria futura.

6.5 R05 - Qualquer cliente trocava a sessão da parede (Alto → Baixo, falta gravar)

- **Antes:** qualquer cliente conectado podia mandar o comando *session* e trocar o identificador que a parede coloca em cada impacto.
- **O que mudou:** a parede atende um aplicativo por vez. O primeiro que pareia com o segredo certo fica com a parede até sair; um segundo aplicativo recebe "parede ocupada" e é desconectado, mesmo com o segredo certo.

  Só o aplicativo pareado manda comandos. Quando ele sai, a parede volta ao identificador da inicialização e fica livre para o próximo.
- **Como testamos:** testes R05 do Anexo B.1, no emulador da parede.
- **O que falta:** gravar o firmware na placa e repetir o teste nela.

6.6 R06 - Login e cadastro sem limite de tentativas (Alto → Baixo)

- **Antes:** era possível tentar senhas sem limite em */auth/sign-in* e criar contas sem limite em */auth/sign-up*.
- **O que mudou:** dois contadores protegem o login e o cadastro. Por endereço IP: 10 tentativas por minuto, com bloqueio de 5 minutos. Por conta (e-mail): 5 tentativas a cada 15 minutos, com bloqueio de 15 minutos.

  Como o bloqueio dura mais que a janela de contagem, quem insiste fica bloqueado por mais tempo. Acima do limite, a API responde 429 com a mensagem "Muitas tentativas. Aguarde alguns minutos e tente de novo.", e a tentativa fica no log de auditoria (R12), sem a senha.
- **Como testamos:** teste R06 do Anexo B.1: cinco logins errados receberam 401, e o sexto e o sétimo, 429.
- **O que falta:** o bloqueio por conta tem um efeito colateral conhecido: quem souber o e-mail de outra pessoa consegue bloqueá-la por 15 minutos errando a senha de propósito. Esse efeito foi registrado e aceito como R25.

6.7 R07 - A parede não limitava clientes nem comandos (Alto → Médio, falta gravar)

- **Antes:** a parede aceitava vários clientes, mensagens de qualquer tamanho e comandos sem limite de frequência. Um cliente que sumia sem fechar a conexão continuava ocupando a parede.
- **O que mudou:** um aplicativo por vez (R05); comandos com mais de 256 bytes são recusados; acima de 20 comandos por segundo, os excedentes são recusados sem ser interpretados.

  O ping do WebSocket, a cada 5 segundos, desconecta o aplicativo que deixa de responder a 2 pings seguidos.
- **Como testamos:** testes R07 do Anexo B.1, no emulador da parede.
- **O que falta:** gravar o firmware na placa e repetir o teste nela. Inundar o rádio Wi-Fi ainda pode atrapalhar a parede; isso não se resolve em software e fica aceito.

6.8 R08 - CORS aberto e API em todas as interfaces (Médio → Baixo)

- **Antes:** a API aceitava requisições de navegador de qualquer origem e escutava em todas as interfaces de rede (*0.0.0.0*).
- **O que mudou:** o CORS fica desligado por padrão, porque o aplicativo mobile não precisa dele, e só é ligado para as origens listadas em *CORS_ORIGINS*. A interface de escuta virou configuração (*HOST*), com padrão *127.0.0.1*, que só aceita conexões da própria máquina.

  Para a rede de testes, o *.env.example* usa *HOST=0.0.0.0* de forma explícita.
- **Como testamos:** teste R08 do Anexo B.1, mais testes de unidade dos valores padrão.

6.9 R09 - Senha padrão do banco como reserva (Alto → Médio)

- **Antes:** sem a variável *DB_PASSWORD*, a API tentava conectar ao banco com um usuário e uma senha conhecidos.
- **O que mudou:** as variáveis *DB_USER* e *DB_PASSWORD* não têm mais valor padrão no código. Sem elas, a API não sobe e avisa qual variável está faltando.
- **Como testamos:** testes de unidade conferem que a API recusa iniciar sem cada uma das duas variáveis.
- **O que falta:** o ambiente publicado precisa de uma conta de banco própria, com senha forte e só as permissões que a aplicação usa (privilégio mínimo).

6.10 R10 - Token de sete dias e sem revogação (Médio → Baixo)

- **Antes:** o token de acesso valia sete dias, e sair da conta não o invalidava: uma cópia do token continuava funcionando.

- **O que mudou:** cada usuário tem uma versão de token no banco, e o token leva a versão em que foi emitido. Ao sair da conta, o aplicativo chama *POST /auth/sign-out*, a versão sobe e todos os tokens anteriores daquele usuário param de funcionar, nesse aparelho e em qualquer outro.

  A validade padrão caiu de 7 dias para 12 horas. O token de acesso também ganhou um público próprio (audience), para que nenhum outro token assinado com o mesmo segredo, como o ticket do R04, seja aceito no lugar dele.

  Não foi preciso criar a renovação de token prevista no Plano: o aplicativo guarda o token só na memória, e o jogador entra de novo a cada vez que abre o app.
- **Como testamos:** teste R10 do Anexo B.1, mais testes de unidade da revogação.

6.11 R11 - Dependências com alertas conhecidos (Alto → Baixo)

- **Antes:** na Entrega 1, o npm audit mostrava 9 alertas no backend e 18 no aplicativo. Até esta revisão, o backend tinha chegado a 11 alertas, um deles crítico.
- **O que mudou:** o backend foi atualizado do NestJS 10 para o 11, e a biblioteca multer, que continuava com um alerta na versão usada pelo NestJS, foi fixada na versão corrigida (2.5.0). No aplicativo, o *npm audit fix* aplicou as atualizações compatíveis.

  O backend ficou sem nenhum alerta. O aplicativo ficou com 5 alertas altos, todos dentro do *@expo/cli*, a ferramenta de linha de comando do Expo, que roda só no computador do desenvolvedor: o node-forge e uma cópia antiga do zod (3.25.76). O zod que o próprio aplicativo usa é o 4.6.7, já corrigido. Essa ferramenta não entra no aplicativo instalado no celular, então esses alertas foram justificados como não alcançáveis, como o Plano previa.
- **Como testamos:** o *npm audit* foi repetido antes e depois (Quadro 8 e Anexo B.6). O pacote Android gerado com *expo export* foi conferido e não contém a biblioteca node-forge, uma das que têm alerta. As suítes de teste do backend e do aplicativo passaram depois das atualizações.
- **Nova rodada (31/10/2026):** o npm audit passou a mostrar 3 alertas moderados no backend e 22 no aplicativo. O código não mudou: o que mudou foi a base de alertas. No aplicativo, as correções compatíveis foram aplicadas e sobraram 21. Eles têm origem em quatro pacotes (a cópia antiga do zod, glob, minimatch e deepmerge), e nenhum dos quatro está no pacote Android (Anexo B.9). No backend, os alertas vêm do glob e do minimatch usados pelo TypeORM, e a correção exige a versão principal 1.x, que fica para depois da apresentação. A API passa entidades e migrações como classes, então o glob não recebe nenhum padrão vindo de fora.

6.12 R12 - Pouca trilha de auditoria (Médio → Baixo)

- **Antes:** eventos de segurança quase não eram registrados, e a parede só escrevia no monitor Serial.
- **O que mudou:** a API escreve uma linha em JSON para cada login, cadastro e logout, com sucesso ou falha, e para cada requisição recusada (400, 401, 403, 409 e 429) ou com erro do servidor. A linha tem o evento, o resultado, o status, o método, a rota, o IP e o usuário, quando houver.

  O e-mail aparece mascarado (*a***@dominio*), e nunca entram senha, token ou o corpo da requisição. Na parede, as recusas de pareamento e os comandos recusados são registrados no Serial.
- **Como testamos:** um login com senha errada e um acesso com token forjado geraram linhas no log, e a senha usada no teste não apareceu em nenhuma delas (Anexo B). Testes de unidade cobrem o conteúdo das linhas.
- **O que falta:** os logs saem no console. Guardá-los e revisá-los fica para o ambiente publicado.

6.13 R17 - Dados pessoais reais em demonstrações (Alto → Médio)

- **Antes:** não havia como criar contas fictícias, e nas demonstrações alguém acabaria usando o próprio e-mail.
- **O que mudou:** dois comandos no backend. *npm run demo:accounts* cria contas fictícias (Jogador Demo 1, 2, 3...) num domínio reservado que nunca pertence a uma pessoa (*demo.smash.test*), com senhas aleatórias mostradas uma única vez. *npm run demo:clean* apaga todas essas contas, junto com as sessões, o progresso e a ofensiva.

  O cadastro continua pedindo só nome, e-mail e senha, o mínimo para a conta funcionar.
- **Como testamos:** duas contas foram criadas e depois apagadas ("2 conta(s) de demonstração apagada(s).").
- **O que falta:** usar os comandos é um procedimento da equipe: criar as contas antes de cada evento e apagá-las depois.

6.14 R23 - A API não enviava cabeçalhos de segurança (novo → Muito Baixo)

- **Antes:** a nova varredura com o OWASP ZAP mostrou que a API não enviava o cabeçalho *X-Content-Type-Options*, revelava o servidor no cabeçalho *X-Powered-By*, permitia guardar em cache respostas com dados do jogador e não isolava a origem contra ataques do tipo Spectre.

- **O que mudou:** a API passou a usar o helmet, que envia os cabeçalhos de segurança e retira o *X-Powered-By*, e toda resposta sai com *Cache-Control: no-store*.
- **Como testamos:** o ZAP foi rodado de novo depois da correção: os quatro avisos sumiram. Sobraram um aviso sobre a versão do próprio ZAP e um informativo, que confirma que as respostas não vão para o cache (Quadro 9 e Anexo B.8).

6.15 R24 - A API aceitava o segredo JWT de exemplo (novo → Médio)

- **Antes:** a revisão do histórico mostrou que o *.env.example* traz um segredo JWT de exemplo, e nada impedia a API de subir com ele. Como esse texto é público, quem o conhece conseguiria forjar o token de qualquer jogador.
- **O que mudou:** a API não sobe se o segredo for o de exemplo ou tiver menos de 32 caracteres, e mostra como gerar um aleatório (*openssl rand -base64 48*). O README da instalação foi atualizado.
- **Como testamos:** a API recusou iniciar com o segredo de exemplo (Anexo B); três testes de unidade cobrem a regra.

6.16 R29 - O emulador comparava o segredo com !== (novo → Muito Baixo)

- **Antes:** o emulador da parede (*scripts/wall-emulator.mjs*) comparava o segredo de pareamento com !==, que para no primeiro caractere diferente. O firmware já não fazia isso, mas o emulador, que deve seguir as mesmas regras, fazia. O achado veio do Aikido (seção 7).
- **O que mudou:** o emulador agora compara com *crypto.timingSafeEqual*, sobre o SHA-256 dos dois valores, para que os tamanhos sejam sempre iguais.
- **Como testamos:** os testes R02 e R05 do Anexo B.1 passaram de novo com o emulador corrigido: segredo ausente ou errado recusado, segredo certo aceito.

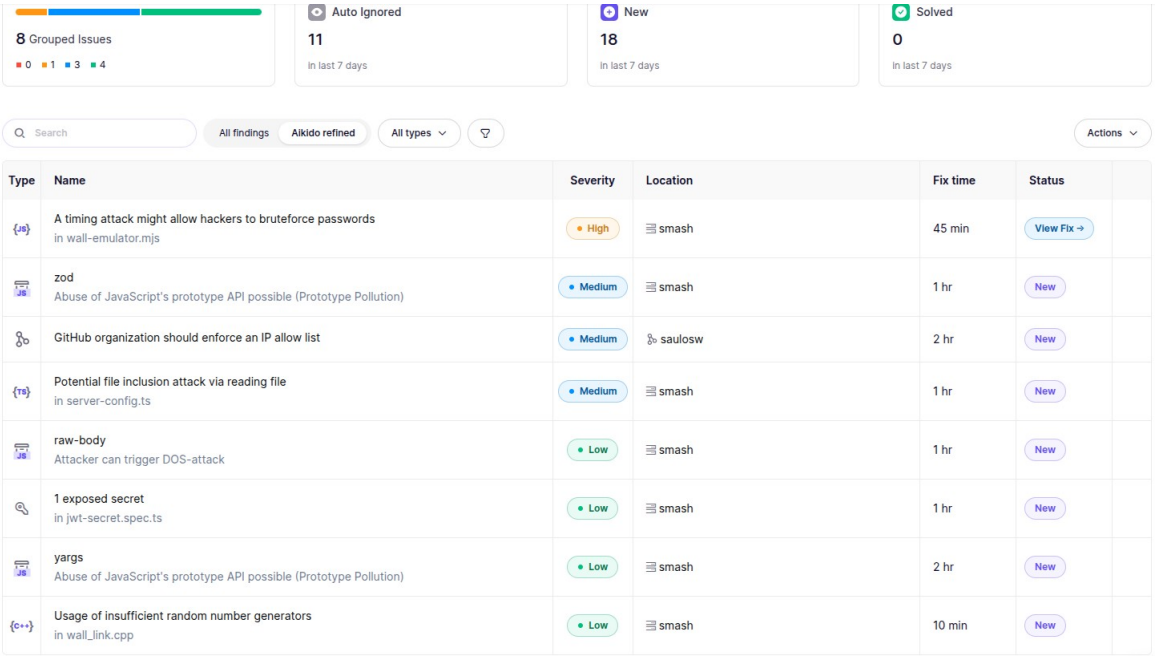
7 ANÁLISE DO REPOSITÓRIO COM O AIKIDO

7.1 Como a análise foi feita

Em aula, o professor indicou plataformas on-line de análise de repositório e pediu a exportação do resultado. A equipe usou o Aikido Security (https://app.aikido.dev), que reúne em uma só análise o código (SAST), as dependências, os segredos e a configuração da conta no GitHub. A análise é de 31/10/2026 e foi feita sobre saulosw/smash, cópia pública do repositório da disciplina que recebe os mesmos commits.

A exportação do relatório é paga, e a equipe usa o plano gratuito. Por isso, a evidência são capturas de tela: a Figura 1 mostra o painel, e o Anexo C traz o detalhe de cada achado.

Figura 1 - Painel do Aikido com os 8 achados agrupados (captura de 31/10/2026).



O painel agrupou 8 achados: nenhum crítico, 1 alto, 3 médios e 4 baixos. A nota da ferramenta é automática, então cada achado foi conferido no código antes de virar trabalho.

7.2 Conferência dos achados

O Quadro 7 mostra o resultado. A coluna "Aikido" traz a severidade e, quando a captura mostra, a nota de risco. Os números de linha são os da análise, antes das correções.

Quadro 7 - Achados do Aikido e tratamento dado pela equipe.

Achado	Aikido	Análise da equipe	Tratamento	Risco
Ataque de tempo na comparação de senha (código) *scripts/wall-emulator.mjs:139-142*	Alto (80)	Real, mas só no emulador, uma ferramenta de desenvolvimento que não guarda dado real. O firmware já comparava sem parar no primeiro caractere (R02).	Corrigido com *timingSafeEqual*	R29
zod 3.25.76, 4 alertas (dependência) *frontend/package-lock.json*	Médio	Não alcançável. A cópia antiga é do @expo/cli. O aplicativo usa o zod 4.6.7, acima da correção (4.6.0), e é só ele que está no pacote Android.	Justificado. A cópia é interna ao Expo	R11
Sem lista de IPs permitidos no GitHub (conta) *saulosw*	Médio (60)	Não se aplica como está: o recurso exige GitHub Enterprise Cloud, e saulosw é uma conta pessoal. O código é público; o que importa é proteger quem tem acesso de escrita.	Aceito, com autenticação em dois fatores	R30
Possível inclusão de arquivo (código) *backend/src/server-config.ts:39*	Médio (45)	Falso positivo. Os caminhos vêm de *TLS_KEY_PATH* e *TLS_CERT_PATH*, definidos por quem opera o servidor e lidos na subida. Nenhuma requisição chega a esse código.	Sem mudança	-
raw-body 3.0.2 (dependência) *backend/package-lock.json*	Baixo (37)	Não alcançável. A falha exige um *limit* inválido, e a API usa o limite padrão, de 100 KB: um corpo de 150 KB recebeu 413 (Anexo B.9).	Justificado. A versão 4.0.0 exige Node 22 e está fora da faixa do body-parser	R11
1 segredo exposto (segredos) *backend/test/unit/ infrastructure/ jwt-secret.spec.ts:5*	Baixo (27)	Falso positivo. Era um valor inventado para o teste, que não aparece em nenhum outro arquivo do repositório nem no .env local.	Teste passou a gerar o valor na hora	-
yargs 17.7.3 (dependência) *frontend/package-lock.json* e *backend/package-lock.json*	Baixo (24)	Não alcançável. Só ferramentas de linha de comando o usam (TypeORM, Metro, React Native, Jest). A API em execução não o carrega, e ele não está no pacote Android (Anexo B.9).	Justificado. A versão 18 é só ESM e vem por ferramentas de terceiros	R11
Gerador aleatório insuficiente (código) *backend/firmware/ smash_wall/ wall_link.cpp:204*	Baixo (15)	Falso positivo no contexto. O valor só identifica a inicialização da parede e não protege nada. No núcleo ESP32 3.3.11, o *random()* já usa o gerador de hardware.	Trocado por *esp_random()*, para deixar isso explícito	-

- **Corrigidos (3):** a comparação do emulador (R29), que era real, e dois falsos positivos em que corrigir custava menos que explicar: o valor fixo do teste e o *random()* do firmware.
- **Não alcançáveis (3):** zod, raw-body e yargs ficam registrados no R11, com a prova de que o trecho vulnerável não é usado pelo sistema em execução.
- **Aceito (1):** a lista de IPs do GitHub (R30), que exige um plano pago e não muda a exposição de um repositório público.

- **Falso positivo sem mudança (1):** a leitura dos arquivos do certificado em *server-config.ts*.

7.3 O que a ferramenta acrescentou

O Aikido apontou itens que as outras ferramentas não marcaram. O Semgrep não indicou a comparação do emulador, o gitleaks não indicou o valor do teste, e o npm audit não lista raw-body nem yargs, que vêm da base própria do Aikido (códigos AIKIDO-2026). Por isso, a análise ficou como mais uma etapa da verificação.

A análise também mostrou o limite de uma nota automática. O achado mais grave do painel estava em uma ferramenta de desenvolvimento, três dos oito eram falsos positivos, e os alertas de dependências só puderam ser descartados depois de conferir o que entra de fato no aplicativo e na API.

8 EVIDÊNCIAS DE TESTES DE SEGURANÇA

Os ataques simulados contra a API e a parede estão no Anexo B.1: cada linha traz o risco, o que foi tentado e o resultado, e o script *npm run security:test* repete todos eles. Os testes da API foram feitos com o backend e o banco locais, e os da parede com o emulador, que segue as mesmas regras do firmware. Os quadros abaixo resumem as dependências, as ferramentas de busca de vulnerabilidades e os testes automatizados.

Quadro 8 - Verificação de dependências (npm audit --omit=dev).

Parte	Entrega 1 (12/09/2026)	Antes das correções (25/10/2026)	Depois das correções (25/10/2026)	Nova rodada (31/10/2026)
Backend	9 alertas	11 (1 crítico, 5 altos e 5 moderados)	0	3 moderados (glob e minimatch, pelo TypeORM)
Aplicativo	18 moderados	16 (1 crítico, 8 altos e 7 moderados)	5 altos, só no @expo/cli, fora do aplicativo instalado	21 (2 altos, 18 moderados e 1 baixo) em ferramentas do Expo e do React Native, fora do aplicativo instalado

Quadro 9 - Nova rodada de busca de vulnerabilidades.

Ferramenta	O que procura	Resultado
gitleaks 8.28	Chaves, tokens e senhas em todos os commits do repositório	O gitleaks não detectou nada nos 82 commits: ele procura chaves e tokens com formato conhecido, e não uma senha comum como smash1234. Por isso o histórico também foi revisado à mão, e essa revisão achou o smash1234 (R03, já conhecido) e o segredo JWT de exemplo, que virou o R24
Semgrep 1.140	Falhas de segurança no código: injeção, JWT, segredos, C/C++	112 regras em 371 arquivos (backend, aplicativo, firmware e scripts): nenhum achado
OWASP ZAP 2.16, baseline	Falhas visíveis nas respostas de uma rota pública	Antes: 0 falhas e 4 avisos de cabeçalhos e cache, que viraram o R23. Depois da correção: 0 falhas e 65 verificações aprovadas; sobraram um aviso sobre a versão do ZAP e um informativo
OWASP ZAP 2.16, API completa	Ataques ativos (injeção, XSS e outros) nas 8 rotas da descrição OpenAPI (o ZAP registrou 12 URLs ao importá-la), autenticado com um token de teste	0 falhas e 114 verificações aprovadas; só o aviso sobre a versão do ZAP. As 1.174 tentativas nas rotas /me foram recusadas pela validação (400). Limitação: nas rotas de login, o limite de tentativas (R06) bloqueou a maior parte dos ataques (429), e a saída da conta ficou fora da varredura para não revogar o token dela
Aikido Security, plano gratuito	Código, dependências, segredos e conta do GitHub	8 achados agrupados (1 alto, 3 médios e 4 baixos): 3 corrigidos, 3 alertas não alcançáveis, 1 aceito e 1 falso positivo sem mudança (seção 7)

Quadro 10 - Testes automatizados no commit de referência.

Parte	Suítes	Testes	Resultado
Backend	15	86	Todos passam
Aplicativo	17	117	Todos passam
Script de segurança	-	21	Todos passam
Firmware	-	Compilação	Compila (73% da memória de programa)

9 RISCOS MATERIALIZADOS E NOVOS RISCOS

9.1 Riscos materializados

- **R03:** a senha antiga da parede foi publicada no repositório público antes da correção, e o histórico do Git não pode ser apagado. A resposta é tratar a senha como vazada e nunca mais usá-la (item 6.3).
- **R11:** entre a Entrega 1 e esta revisão surgiram novos alertas de dependências, incluindo um crítico no backend. Isso confirma que a verificação precisa ser repetida a cada entrega, e não feita uma única vez. A rodada de 31/10/2026 mostrou o mesmo: surgiram alertas novos em menos de uma semana, sem mudança no código (item 6.11).
- **R15:** o risco original era não ter tempo de implementar as respostas até a Entrega 2. Elas foram implementadas, então ele foi reformulado para o que sobrou: concluir até a apresentação, de 17 a 24/11/2026, os passos que dependem da placa e do servidor.

9.2 Novos riscos

As correções criaram ou revelaram onze riscos novos: três vieram das próprias correções (R20 a R22), dois da nova rodada de busca de vulnerabilidades (R23 e R24), três de efeitos colaterais das correções e da autenticação (R25 a R27), um da revisão dos requisitos de privacidade do enunciado (R28) e dois da análise com o Aikido (R29 e R30). O Quadro 11 os classifica com as mesmas escalas da Entrega 1, e a resposta de cada um está no Quadro 4.

Quadro 11 - Novos riscos identificados na Entrega 2.

ID	Risco	Categoria	Prob.	Impacto	Severidade
R20	O segredo de pareamento vai dentro do pacote do aplicativo (*EXPO_PUBLIC_DEVICE_TOKEN*) , e quem tiver o arquivo do app consegue extraí-lo.	Técnico	Baixa	Médio	Baixo

ID	Risco	Categoria	Prob.	Impacto	Severidade
R21	Versões antigas do aplicativo deixam de salvar sessões, porque o servidor passou a exigir o ticket (R04).	Técnico	Média	Baixo	Médio
R22	Atrás de um proxy HTTPS, a limitação de tentativas veria todos os aparelhos com o mesmo IP e bloquearia todos juntos.	Técnico	Média	Médio	Médio
R23	A API não enviava cabeçalhos de segurança e permitia guardar respostas em cache (achado do ZAP).	Técnico	Muito Baixa	Baixo	Muito Baixo
R24	A API subia com o segredo JWT de exemplo do repositório, o que permitiria forjar tokens (achado na revisão do histórico).	Técnico	Muito Baixa	Muito Alto	Médio
R25	Quem souber o e-mail de outra pessoa consegue bloquear o login dela por 15 minutos errando a senha de propósito (efeito do R06).	Técnico	Média	Baixo	Médio
R26	O segredo de pareamento vai no endereço da conexão (*?token=*), em texto claro, e pode aparecer em registros de rede.	Técnico	Baixa	Médio	Baixo
R27	O cadastro responde que o e-mail já está em uso, o que revela quem tem conta no Smash.	Técnico	Média	Baixo	Médio
R28	O jogador não consegue apagar a própria conta e seus dados pelo aplicativo, um direito do titular na LGPD.	Organizacional	Média	Médio	Médio
R29	O emulador da parede comparava o segredo de pareamento com !==, sujeito a ataque de tempo (achado do Aikido).	Técnico	Muito Baixa	Baixo	Muito Baixo
R30	A conta do GitHub não restringe o acesso por lista de IPs permitidos (achado do Aikido).	Técnico	Baixa	Médio	Baixo

10 COBERTURA DOS REQUISITOS DE SEGURANÇA

A seção 5 do enunciado do Projeto Interdisciplinar pede proteção contra o OWASP Top 10, cibersegurança do dispositivo IoT, atualização controlada e privacidade conforme a LGPD. O Quadro 12 liga os riscos deste relatório às categorias do OWASP Top 10 (2021) e do OWASP IoT Top 10 (2018), para mostrar o que já é coberto e o que fica fora.

Quadro 12 - Riscos do Smash nas categorias OWASP.

Categoria OWASP	Riscos e tratamento no Smash
A01 Controle de acesso quebrado	R04, R05 e R10: sessão aberta pelo servidor, um app pareado e revogação de token
A02 Falhas de criptografia	R01 (HTTPS) e R24 (segredo JWT forte)
A03 Injeção	Entradas validadas com lista de campos permitidos e consultas parametrizadas; Semgrep sem achados
A04 Design inseguro	R04: o resultado deixou de ser só declarado pelo aplicativo
A05 Configuração insegura	R08, R09, R23 e R30
A06 Componentes vulneráveis	R11
A07 Falhas de identificação e autenticação	R06, R10, R25, R27 e R29
A08 Falhas de integridade de software e dados	R04 e atualização do firmware só por cabo (item 10.1)
A09 Falhas de registro e monitoramento	R12
A10 SSRF	Não se aplica: a API não busca endereços externos
I1 Senhas fracas, previsíveis ou fixas no código	R03 e R24
I2 Serviços de rede inseguros	R02 e R07
I3 Interfaces do ecossistema inseguras	R01, R04 e R08
I4 Falta de atualização segura	Atualização só por cabo USB (item 10.1)
I5 Componentes inseguros ou desatualizados	R11
I6 Proteção de privacidade insuficiente	R17 e R28 (item 10.2)
I7 Transferência e armazenamento inseguros	R01, R02 e R26
I8 Falta de gestão do dispositivo	R05 e R18
I9 Configurações padrão inseguras	R03, R09 e R24
I10 Falta de proteção física	Placa e porta USB acessíveis: aceito, porque a parede só funciona em ambiente supervisionado

10.1 Atualização controlada do firmware

O firmware só pode ser atualizado por cabo USB, com o computador ligado à placa. Não existe atualização pela rede (OTA), então ninguém consegue trocar o

firmware a distância. O controle é físico: só a equipe grava a placa, e o quadro *hello* de cada conexão informa a versão do protocolo, o que faz o aplicativo detectar e avisar quando a placa está com um firmware antigo ("Parede incompatível"). Uma atualização pela rede, com firmware assinado, fica fora do MVP.

10.2 Privacidade, LGPD e proteção de menores

O Smash coleta só o necessário para a conta funcionar: nome, e-mail, senha (guardada apenas como hash bcrypt) e o histórico de treinos. Não coleta idade, documento, localização nem imagem, e a parede não tem câmera. Nas apresentações, só são usadas contas fictícias (R17).

O MVP não tem perfis de crianças e não pede a idade do jogador. Se o Smash passar a atender menores, a LGPD (art. 14) exige o consentimento de um dos pais ou responsável, coleta mínima e acesso restrito, como o enunciado pede; isso fica fora do MVP e é condição para abrir o produto a esse público. O direito de o titular apagar os próprios dados ainda não tem tela no aplicativo e está registrado como R28.

11 LIMITAÇÕES DA VERIFICAÇÃO

Esta seção registra o que não pôde ser comprovado nesta entrega.

- **Placa física:** os testes da parede rodaram no emulador. O firmware novo compila, mas ainda não foi gravado na placa (R02, R03, R05 e R07).
- **Ambiente publicado:** os testes usaram a API e o banco locais, e o HTTPS foi testado com certificado de teste, não com um certificado reconhecido (R01).
- **OWASP ZAP:** o limite de tentativas bloqueou a maior parte dos ataques nas rotas de login, e a saída da conta ficou fora da varredura (Quadro 9).
- **Aikido:** o relatório não foi exportado, porque a exportação é paga; a evidência são as capturas de 31/10/2026. Os 11 itens que a ferramenta ignorou sozinha não foram analisados. As correções da seção 7 só aparecem no painel depois de uma nova análise do repositório.
- **npm audit:** o resultado muda com a data, porque a base de alertas muda. Os números valem para o dia indicado (Quadro 8).

12 RISCO RESIDUAL E PRÓXIMOS PASSOS

Depois dos passos que faltam, nenhum risco fica Extremo, e só o R14, que é físico, continua Alto. Antes da apresentação na Semana FECAP de Tecnologia (17 a 24/11/2026), falta:

- **Publicar a API com HTTPS (R01, R22):** servidor com certificado reconhecido, *TRUST_PROXY=true* se ele estiver atrás de um proxy, e *EXPO_PUBLIC_API_URL* com *https://* no aplicativo.
- **Gravar o firmware novo na placa (R02, R03, R05 e R07):** criar o *secrets.h* com senha do Wi-Fi e segredo de pareamento novos, colocar o mesmo segredo em *EXPO_PUBLIC_DEVICE_TOKEN* no aplicativo e rodar *npm run security:test* apontando para a placa, para repetir nela os testes da parede do Anexo B.1.
- **Preparar o banco do ambiente publicado (R09):** conta própria, senha forte e só as permissões que a aplicação usa.
- **Usar contas fictícias nas demonstrações (R17):** *npm run demo:accounts* antes do evento e *npm run demo:clean* depois.
- **Repetir o npm audit e a análise do Aikido (R11)** antes da apresentação e registrar o resultado.
- **Continuar a validação física (R14)** com a parede montada: um alvo, dois alvos lado a lado e depois os seis.
- **Criar a exclusão da conta pelo jogador (R28).**
- **Proteger o acesso ao repositório (R30):** autenticação em dois fatores nas contas com acesso de escrita.

Os responsáveis seguem as trilhas do Quadro 2 da Entrega 1: Firmware, Backend e Aplicativo, com a revisão do registro conduzida pelo Gerente de Projeto.

13 MONITORAMENTO E REVISÃO

O registro segue a rotina do Plano da Entrega 1: revisão semanal na reunião da equipe, antes de qualquer demonstração da parede e a cada marco do projeto. O próximo marco é a apresentação na Semana FECAP de Tecnologia, de 17 a 24/11/2026.

O status de um risco só muda com evidência que pode ser repetida: commit, configuração ou teste. Antes da apresentação, a equipe repete *npm run security:test*, o npm audit e a análise do Aikido e registra o resultado. Riscos novos entram no registro a partir do R31.

APROVAÇÃO

Quadro 13 - Aprovação do relatório.

Nome	Cargo	Assinatura	Data
Saulo Pereira de Jesus	Gerente do Projeto	Saulo Pereira de Jesus	06/11/2026
Felipe Vallim Soares	Responsável - Firmware (ESP32)	Felipe Vallim Soares	06/11/2026
Guilhermy Mariano Lisboa Garcia	Responsável - Backend (API e banco)	Guilhermy Mariano Lisboa Garcia	06/11/2026
Gustavo Oliveira Demetrio	Responsável - Aplicativo Mobile	Gustavo Oliveira Demetrio	06/11/2026
Prof. Victor Bruno Alexander Rosetti	Patrocinador do Projeto		

ANEXO A - REGISTRO DE RISCOS ATUALIZADO

O Quadro 14 traz os 19 riscos da Entrega 1 e os onze novos, com a severidade antes e depois das correções. O Quadro 15 liga cada correção ao commit que a implementa e aos arquivos onde ela pode ser conferida. Todos os commits citados estão no repositório público.

Quadro 14 - Registro de riscos atualizado (severidade na Entrega 1 e severidade residual).

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R01	A API usa HTTP sem criptografia e o aplicativo envia o token JWT nesse canal.	Extremo	Concluído no código; falta publicar	Baixa	Muito Alto	Médio
R02	O WebSocket da parede usa ws:// e aceita qualquer cliente sem autenticação.	Extremo	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R03	A senha do SoftAP é previsível e está versionada em repositório público.	Extremo	Concluído no código; falta gravar a placa	Muito Baixa	Alto	Baixo
R04	O resultado da sessão é declarado pelo aplicativo e aceito sem prova de que a partida ocorreu.	Alto	Concluído	Média	Médio	Médio
R05	Qualquer cliente conectado pode trocar o identificador de sessão da parede.	Alto	Concluído no código; falta gravar a placa	Baixa	Médio	Baixo
R06	Login e cadastro não têm limitação de tentativas.	Alto	Concluído	Baixa	Médio	Baixo
R07	O ESP32 não limita clientes, tamanho de mensagens ou frequência de comandos.	Alto	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R08	A API aceita qualquer origem CORS e escuta em todas as interfaces.	Médio	Concluído	Muito Baixa	Médio	Baixo
R09	O banco usa usuário e senha padrão de desenvolvimento como reserva.	Alto	Concluído	Baixa	Muito Alto	Médio
R10	O JWT dura sete dias e não possui revogação no servidor.	Médio	Concluído	Baixa	Médio	Baixo
R11	As dependências possuem alertas conhecidos no npm audit.	Alto	Concluído	Baixa	Médio	Baixo
R12	Eventos de segurança têm pouca trilha de auditoria.	Médio	Concluído	Baixa	Baixo	Baixo
R13	A integração MQTT prevista não tem controles de segurança definidos.	Alto	Encerrado: MQTT fora do escopo	-	-	-

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R14	Isolamento mecânico insuficiente faz a parede registrar o alvo errado.	Alto	Em andamento	Média	Alto	Alto
R15	Tempo insuficiente para concluir os passos pendentes até a apresentação (17 a 24/11). Reformulado.	Alto	Em andamento	Baixa	Alto	Médio
R16	Alteração de critérios, datas ou formatos das entregas.	Médio	Em andamento	Média	Médio	Médio
R17	Dados pessoais reais usados em testes e demonstrações.	Alto	Concluído	Baixa	Alto	Médio
R18	Falha ou dano da única placa ESP32 disponível.	Médio	Em andamento	Baixa	Alto	Médio
R19	Rede Wi-Fi do local de demonstração indisponível ou instável.	Médio	Encerrado	Muito Baixa	Médio	Baixo
R20	Segredo de pareamento dentro do pacote do aplicativo.	-	Em andamento	Baixa	Médio	Baixo
R21	Versões antigas do aplicativo deixam de salvar sessões.	-	Concluído	Média	Baixo	Médio
R22	Limitação de tentativas atrás de um proxy HTTPS.	-	Concluído	Média	Médio	Médio
R23	API sem cabeçalhos de segurança e com respostas guardáveis em cache.	-	Concluído	Muito Baixa	Baixo	Muito Baixo
R24	API aceitava o segredo JWT de exemplo.	-	Concluído	Muito Baixa	Muito Alto	Médio
R25	Bloqueio de login de outra pessoa pelo limite de tentativas.	-	Aceito	Média	Baixo	Médio
R26	Segredo de pareamento no endereço da conexão, em texto claro.	-	Aceito	Baixa	Médio	Baixo
R27	Cadastro revela se o e-mail já tem conta.	-	Aceito	Média	Baixo	Médio
R28	Sem exclusão da conta pelo próprio jogador (LGPD).	-	Não iniciado	Média	Médio	Médio
R29	O emulador comparava o segredo de pareamento com !==.	-	Concluído	Muito Baixa	Baixo	Muito Baixo
R30	Conta do GitHub sem lista de IPs permitidos.	-	Aceito	Baixa	Médio	Baixo

Quadro 15 - Commit e arquivos de cada correção (caminhos relativos a src/Entrega1).

ID	Commit	Arquivos
R01	810a16b e c98a80b	*backend/src/server-config.ts:24-42*, *backend/src/main.ts:21-43* e *frontend/src/config/env.ts:30-35*
R02	3946d66	*backend/firmware/smash_wall/wall_link.cpp:37-61* e *97-108*; *frontend/src/device/adapters/websocket/wall-protocol.ts:53-72*

ID	Commit	Arquivos
R03	3946d66 e 38529e3	*backend/firmware/smash_wall/secrets_example.h*, *backend/firmware/smash_wall/wall_link.cpp:10-13* e *backend/firmware/smash_wall/config.h:33*
R04	3b5380b	*backend/src/application/use-cases/start-training-session.use-case.ts:34*, *backend/src/application/use-cases/record-training-session.use-case.ts:83-97*, *backend/src/domain/training-session/training-session.entity.ts:72-80*, *backend/src/infrastructure/database/migrations/1757310000000-AddSessionTicket.ts:15* e *frontend/src/gameplay/services/sessionService.ts:64*
R05	3946d66	*backend/firmware/smash_wall/wall_link.cpp:74-83* e *110-118*
R06	9a8417b	*backend/src/presentation/http/auth-throttling.ts:15-32* e *backend/src/presentation/http/auth.controller.ts:13*
R07	3946d66	*backend/firmware/smash_wall/config.h:36-50* e *backend/firmware/smash_wall/wall_link.cpp:85-96*, *120-144* e *221*
R08	810a16b	*backend/src/server-config.ts:24-42* e *backend/.env.example*
R09	da16065	*backend/src/infrastructure/database/database.config.ts:16-32*
R10	e8a8476	*backend/src/application/use-cases/verify-access-token.use-case.ts:14*, *backend/src/application/use-cases/sign-out.use-case.ts:7*, *backend/src/infrastructure/infrastructure.module.ts:82*, *backend/src/presentation/http/auth.controller.ts:33* e *frontend/src/store/authStore.ts:43*
R11	b59160e e 3ce31d4	*backend/package.json* (versão fixada do multer), *backend/package-lock.json* e *frontend/package-lock.json*
R12	9b5698f	*backend/src/presentation/http/security-audit.ts:35-99* e *backend/src/app.module.ts:28*
R13	cb9f2ac	MQTT retirado do escopo do projeto
R17	a4cfb7e	*backend/src/infrastructure/database/demo-accounts.ts:18* e *backend/package.json:14-15*
R21	3b5380b	*backend/src/presentation/http/dto/record-session.dto.ts*
R22	9a8417b	*backend/src/server-config.ts:40* e *backend/src/main.ts:33-35*
R23	4e308b6	*backend/src/main.ts:26-32*
R24	a27d541	*backend/src/infrastructure/security/jwt-secret.ts* e *backend/src/infrastructure/infrastructure.module.ts:81* e *89*
Script	15a85cb e 9d5e68e	*scripts/security-tests.mjs* (*npm run security:test*) e *scripts/zap-openapi.yaml*
R29	Tag entrega2-ciberseguranca-aikido	*scripts/wall-emulator.mjs:20, 91-95 e 146*
Aikido	Tag entrega2-ciberseguranca-aikido	*backend/test/unit/infrastructure/jwt-secret.spec.ts:1 e 7, backend/firmware/smash_wall/wall_link.cpp:6 e 205 e frontend/package-lock.json*

ANEXO B - SAÍDAS DOS TESTES

As saídas abaixo foram copiadas do terminal nas execuções feitas para este relatório. Só foram retiradas as cores, os horários e as linhas repetidas. Os endereços são da máquina de testes (127.0.0.1), e qualquer pessoa pode repetir os testes com os comandos indicados.

B.1 Script de testes de segurança

Comando: *npm run security:test*, com a API e o emulador da parede no ar.

```
API http://127.0.0.1:3990 · parede ws://127.0.0.1:8282


[OK]     R06  7 logins com senha errada na mesma conta -> 401, 401, 401, 401, 401, 429, 429
[OK]     R27  6 logins num e-mail que não tem conta -> 401, 401, 401, 401, 401, 429 (igual a uma
conta real)
[OK]     R27  cadastrar um e-mail que já existe -> 409 "Este e-mail já está em uso." (risco
aceito)
[OK]     R10  usar o token depois de sair da conta -> antes 200, saída 204, depois 401
[OK]     R04  começar o nível 5 sem ter passado do 4 -> 400 "Sessão de treino inválida: nível
ainda bloqueado."
[OK]     R04  enviar um resultado legítimo -> 201, 120 XP
[OK]     R04  enviar o mesmo resultado de novo -> 409 "Esta sessão de treino já foi registrada."
[OK]     R04  enviar resultado sem comprovante -> 400 "A sessão precisa ser iniciada antes de ser
registrada."
[OK]     R04  declarar 1 hora de jogo logo depois de começar -> 400 "Sessão de treino inválida:
duração maior que o tempo desde o início."
[OK]     R04  trocar o nível no resultado -> 400 "Sessão de treino inválida: modo ou nível
diferente do iniciado."
[OK]     R04  declarar 999.999 pontos com 6 acertos -> 400 "Sessão de treino inválida: pontuação
acima do máximo possível."
[OK]     R04  usar o comprovante como token de acesso -> 401
[OK]     R08  chamar a API a partir de um site qualquer -> sem permissão
[OK]     R23  conferir os cabeçalhos de segurança -> nosniff=nosniff, x-powered-by=ausente,
cache=no-store
[OK]     R02  conectar à parede sem senha de pareamento -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha de pareamento errada -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha certa -> hello
[OK]     R05  conectar um segundo aplicativo com a senha certa -> "parede ocupada", fechada: true
[OK]     R07  mandar uma mensagem de 400 bytes -> "comando grande demais"
[OK]     R07  mandar 30 comandos de uma vez -> 20 aceitos, 10 recusados
[OK]     R05  parear outro aplicativo depois que o primeiro saiu -> hello


21 de 21 testes passaram.
```

B.2 HTTPS com certificado de teste (R01)

```
$ curl -sk https://127.0.0.1:3989/rankings/xp -o /dev/null -w "%{http_code}\n"
200
$ curl -s http://127.0.0.1:3989/rankings/xp
000
(exit 52)
```

O código 000 e a saída 52 do curl indicam que a API não respondeu por HTTP na porta que serve HTTPS.

B.3 Linha do registro de auditoria após um login errado (R12)

```
{"level":"warn","pid":106119,"timestamp":1792984442933,"message":
{"event":"auth.sign_in","outcome":"failure","status":401,"method":"POST","path":"/auth/sign-
in","ip":"127.0.0.1","email":"a***@demo.smash.test"},"context":"Security"}
```

B.4 Firmware compilado sem o secrets.h (R03)

```
$ arduino-cli compile --fqbn esp32:esp32:esp32 smash_wall   (sem secrets.h)
smash_wall/wall_link.cpp:10:2: error: #error "Falta secrets.h: copie secrets_example.h para
secrets.h e troque as senhas."
   10 | #error "Falta secrets.h: copie secrets_example.h para secrets.h e troque as senhas."
```

B.5 API iniciada com o segredo JWT de exemplo (R24)

```
$ JWT_SECRET=troque-este-segredo-em-producao node dist/main.js
Error: JWT_SECRET fraco: use um valor aleatório com 32 caracteres ou mais (openssl rand -base64
48).
```

B.6 npm audit --omit=dev, antes e depois (R11)

```
$ cd backend && npm audit --omit=dev   # antes: commit 5444765
11 vulnerabilities (5 moderate, 5 high, 1 critical)
$ cd backend && npm audit --omit=dev   # depois: tag entrega2-ciberseguranca-final
found 0 vulnerabilities
$ cd frontend && npm audit --omit=dev   # antes: commit 5444765
16 vulnerabilities (7 moderate, 8 high, 1 critical)
$ cd frontend && npm audit --omit=dev   # depois: tag entrega2-ciberseguranca-final
node-forge  *
node-forge RSA PKCS#1 v1.5 signature verification accepts extra nested DigestAlgorithm elements -
https://github.com/advisories/GHSA-86w9-cpqp-85rv
zod  <4.6.6
5 high severity vulnerabilities

$ cd backend && npm audit --omit=dev   # 31/10/2026
3 moderate severity vulnerabilities
$ cd frontend && npm audit --omit=dev   # 31/10/2026, antes de aplicar o npm audit fix
22 vulnerabilities (1 low, 18 moderate, 3 high)
$ cd frontend && npm audit --omit=dev   # 31/10/2026, depois do npm audit fix
21 vulnerabilities (1 low, 18 moderate, 2 high)
```

B.7 Testes automatizados (npm test)

```
$ npm --prefix backend test
Test Suites: 15 passed, 15 total
Tests:       86 passed, 86 total
$ npm --prefix frontend test
Test Suites: 17 passed, 17 total
Tests:       117 passed, 117 total
```

B.8 Busca de vulnerabilidades (Quadro 9)

```
$ docker run ghcr.io/gitleaks/gitleaks:v8.28.0 git /repo --redact
INF 82 commits scanned.
INF scanned ~1942510 bytes (1.94 MB) in 600ms
INF no leaks found
$ docker run semgrep/semgrep:1.140.0 semgrep scan --config p/typescript --config p/nodejs \
      --config p/javascript --config p/secrets --config p/jwt --config p/c ...
Ran 112 rules on 371 files: 0 findings.
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-baseline.py -t http://127.0.0.1:3992/rankings/xp
# antes da correção
WARN Insufficient Site Isolation Against Spectre Vulnerability [90004] risco=Low
WARN Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037] risco=Low
WARN X-Content-Type-Options Header Missing [10021] risco=Low
WARN ZAP is Out of Date [10116] risco=Low
WARN Storable and Cacheable Content [10049] risco=Informational
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-baseline.py -t http://127.0.0.1:3992/rankings/xp
# depois da correção
WARN ZAP is Out of Date [10116] risco=Low
WARN Non-Storable Content [10049] risco=Informational
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-api-scan.py -t zap-openapi.yaml -f openapi \
      -O http://127.0.0.1:3992 -z "<cabeçalho Authorization com token de teste>"   # todas as
rotas
Number of Imported URLs: 12
Total of 38 URLs
FAIL-NEW: 0    FAIL-INPROG: 0  WARN-NEW: 1     WARN-INPROG: 0  INFO: 0 IGNORE: 0       PASS: 114
WARN-NEW: ZAP is Out of Date [10116]
```

B.9 Conferências da análise do Aikido (31/10/2026)

Comandos usados para conferir os achados do Quadro 7, já com as correções aplicadas.

```
$ curl -X POST .../auth/sign-in    # limite de corpo da API (raw-body)
corpo 150 KB -> 413
corpo 90 KB -> 400
$ node -e "require('typeorm'); ..."    # módulos carregados pela API em execução
yargs carregado pelo typeorm em runtime: false
$ npx expo export --platform android --source-maps    # módulos no mapa do pacote Android
node-forge, yargs, glob, minimatch, deepmerge: 0 módulos
zod: 95 módulos, todos em node_modules/zod (4.6.7); cópia em expo/node_modules/zod: 0 módulos
@expo/cli: 1 módulo (build/metro-require/require.js, o carregador do Metro)
$ npm run security:test    # com o emulador corrigido
21 de 21 testes passaram.
$ npm --prefix backend test
Tests:       86 passed, 86 total
$ npm --prefix frontend test
Tests:       117 passed, 117 total
$ arduino-cli compile --fqbn esp32:esp32:esp32 smash_wall
Sketch uses 968003 bytes (73%) of program storage space. Maximum is 1310720 bytes.
```

ANEXO C - CAPTURAS DO AIKIDO

A exportação do relatório é paga, então as telas abaixo fazem esse papel. Cada uma mostra o detalhe de um achado do Quadro 7, como o Aikido o apresentou em 31/10/2026, antes das correções. A própria ferramenta mascara o valor do segredo na Figura 7.

Figura 2 - Ataque de tempo em wall-emulator.mjs.

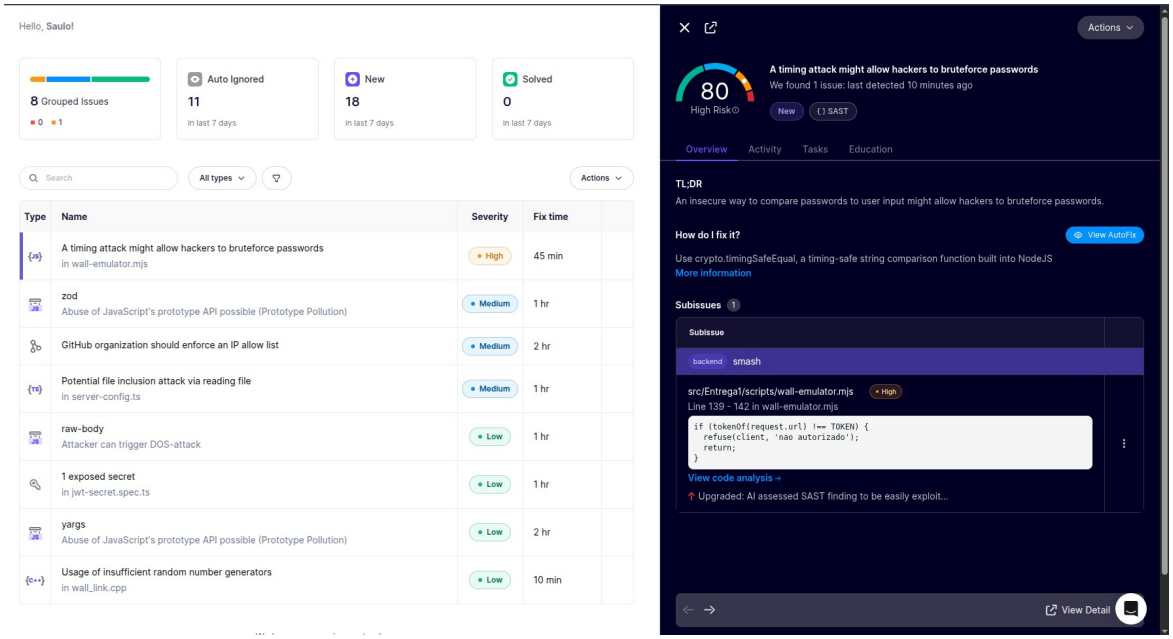


Figura 3 - Alertas do zod.

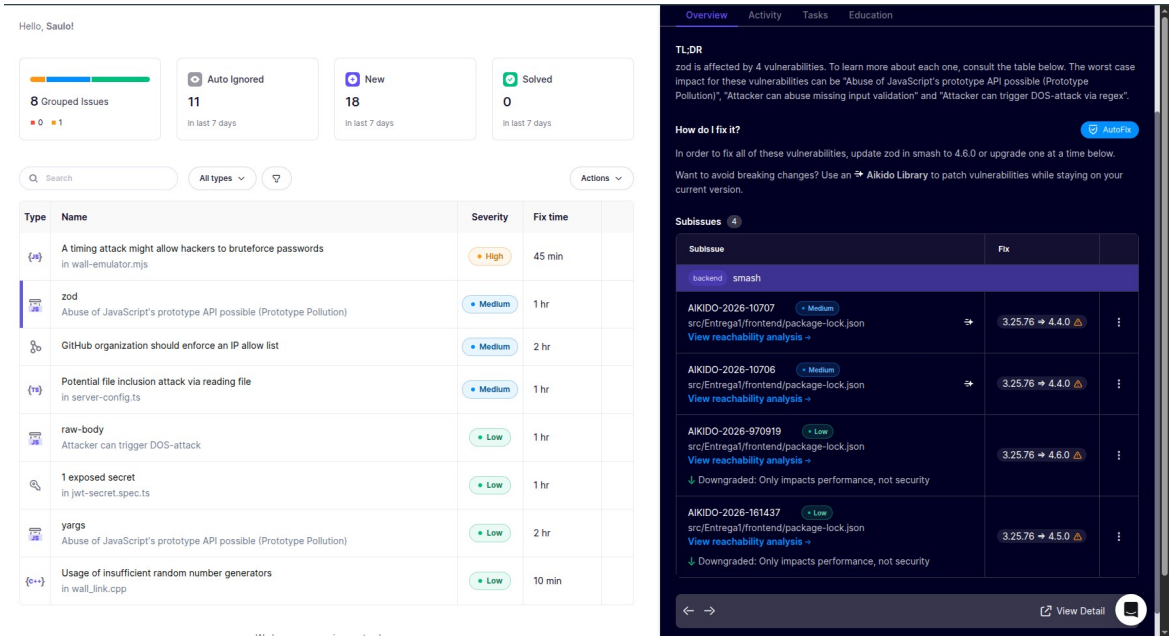


Figura 4 - Lista de IPs permitidos no GitHub.

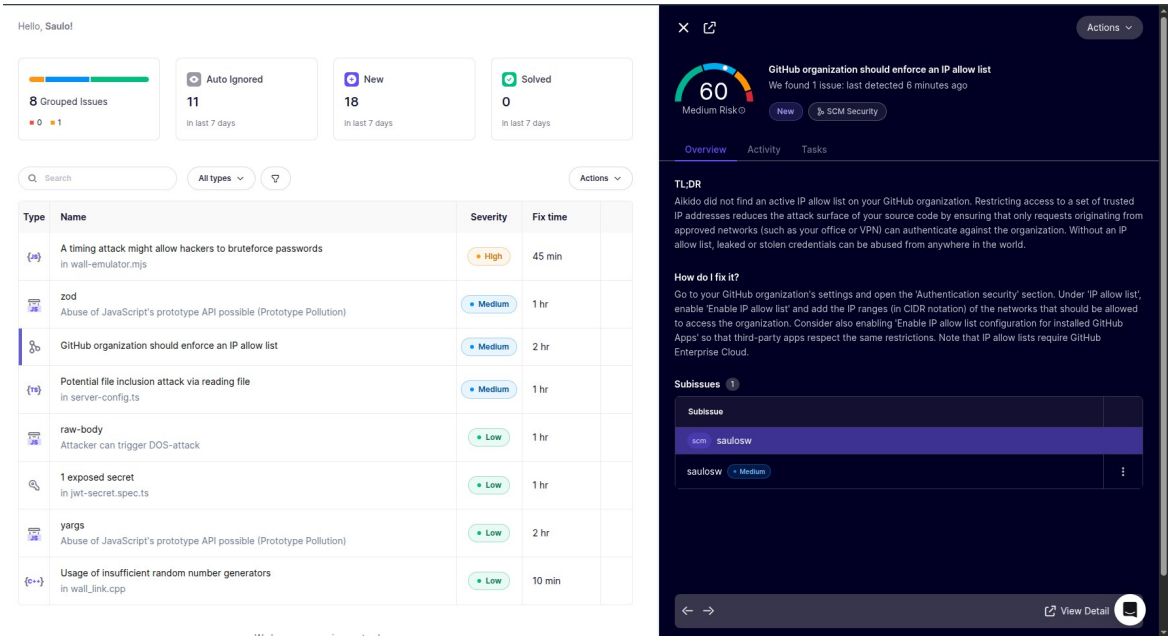


Figura 5 - Possível inclusão de arquivo em server-config.ts.

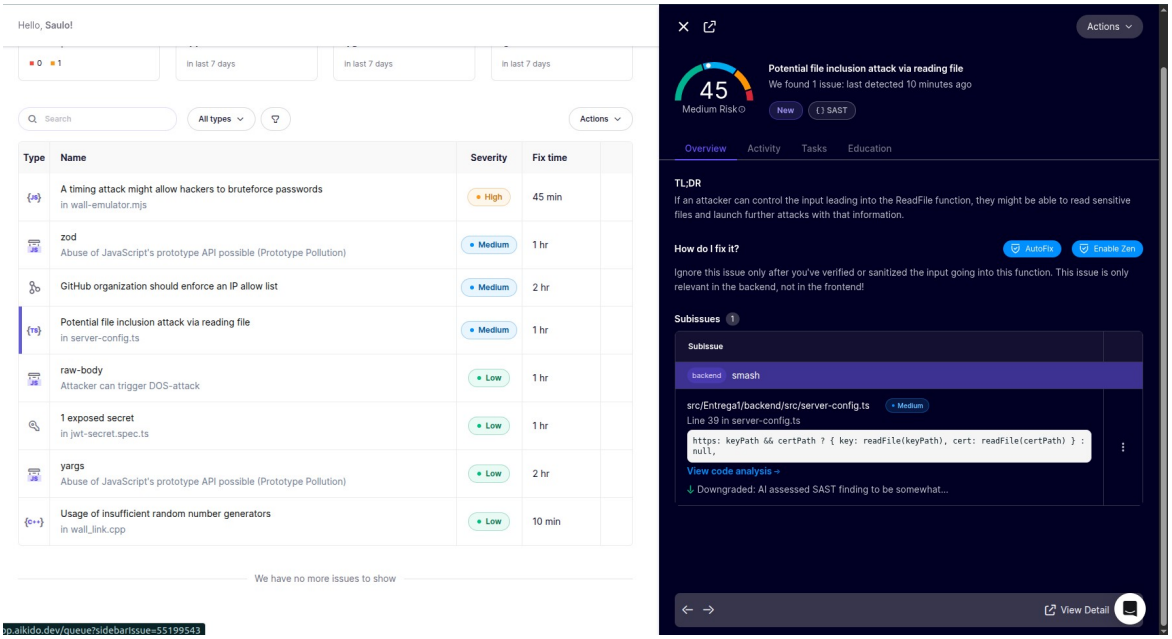


Figura 6 - Alerta do raw-body.

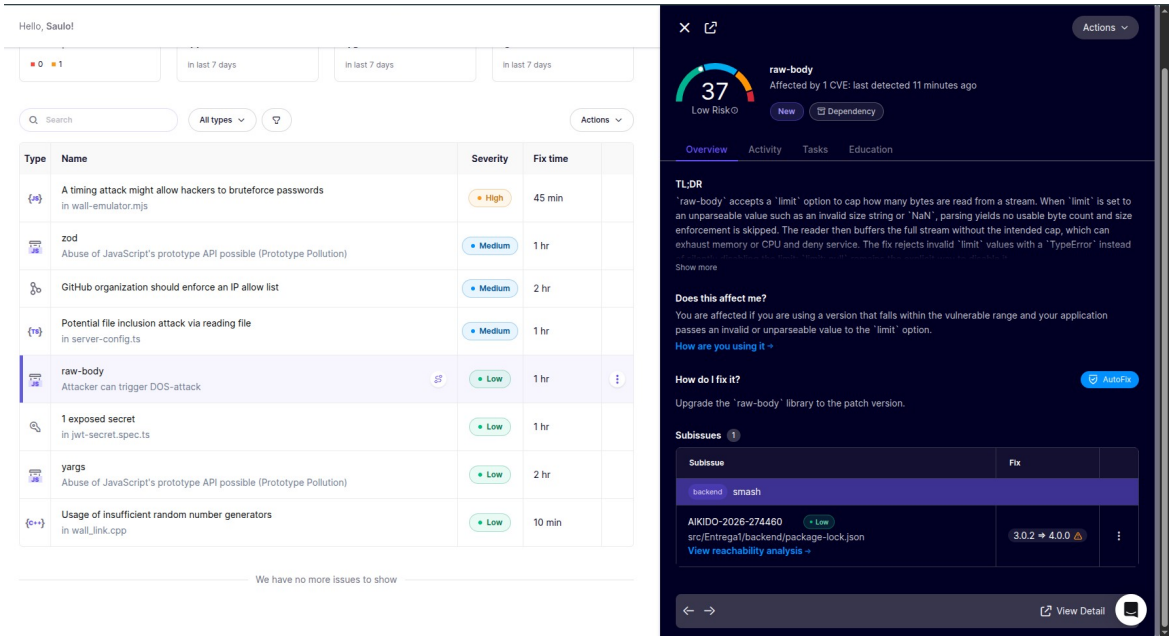


Figura 7 - Segredo em jwt-secret.spec.ts.

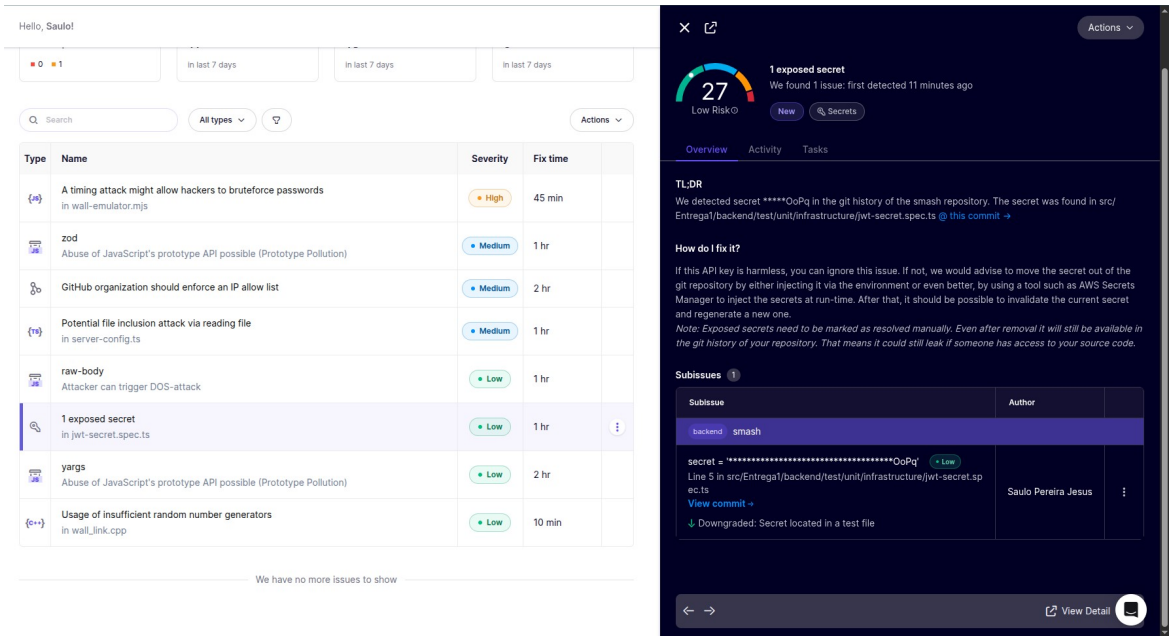


Figura 8 - Alerta do yargs.

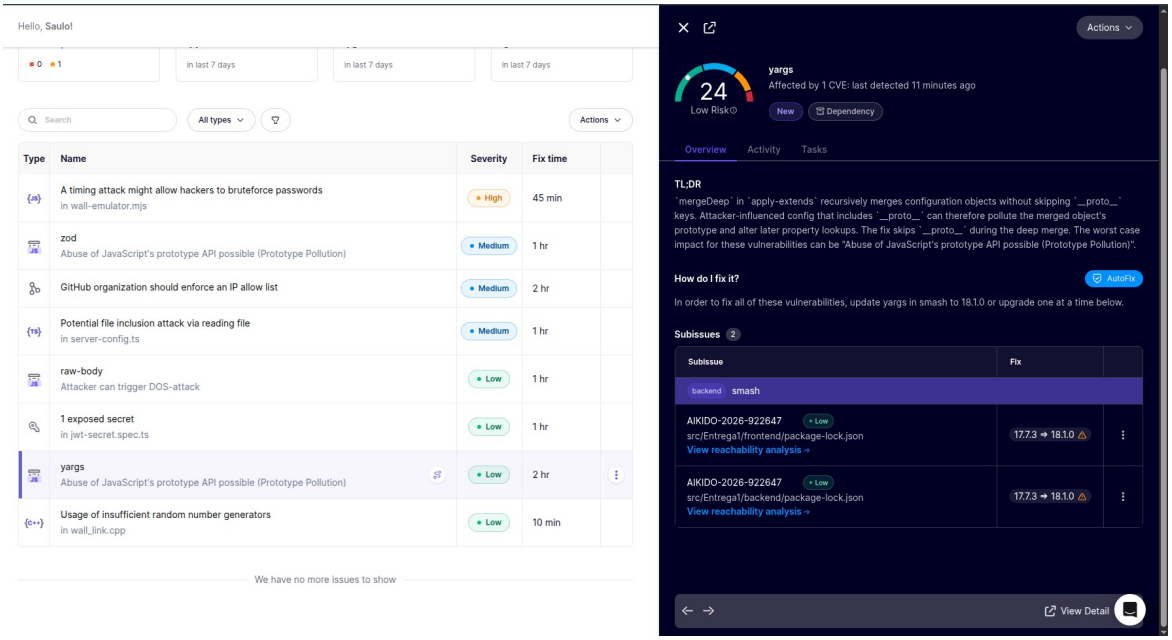


Figura 9 - Gerador aleatório em wall_link.cpp.

